MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH & STUDIES

SCHOOL OF COMPUTER APPLICATIONS

PRACTICAL FILE

DATA STRUCTURE AND ALGORITHM

ACADEMIC SESSION: 2026-27

SUBMITTED TO

Mr. Sachin Sharma

Faculty / Assistant Professor

School of Computer Applications

MRIIRS

SUBMITTED BY

Neha Sharma

Roll No. : 26MCAP101N047

Course : MCA

Semester : 1

1. **Write a program to perform various operations in the list:**

-insertion

- Deletion

- Display

INPUT:-

```
53  list = []
54  while True:
55      print("\n1. Insertion")
56      print("2. Deletion")
57      print("3. Display")
58      print("4. Exit")
59
60      choice = int(input("Enter your choice: "))
61
62      if choice == 1:
63          num = int(input("Enter the number to insert: "))
64          list.append(num)
65          print("Number inserted successfully")
66
67      elif choice == 2:
68          num = int(input("Enter the number to delete: "))
69          if num in list:
70              list.remove(num)
71              print("Number deleted successfully")
72          else:
73              print("Number not found in the list")
74
75      elif choice == 3:
76          print("List:", list)
77
78      elif choice == 4:
79          print("Program ended")
80          break
81
82      else:
83          print("Invalid choice")
```

OUTPUT:-

```
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the number to insert: 23
Number inserted successfully

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the number to insert: 34
Number inserted successfully

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 2
Enter the number to delete: 34
Number deleted successfully

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3
List: [23]

1. Insertion
2. Deletion
3. Display
```

2. Write a Program to sort the list using: - Quick Sort - Merge Sort . Also, find the comparison on the basis of time complexity.

INPUT:-

```python
# Quick Sort
def quick_sort(list):
    if len(list) <= 1:
        return list

    pivot = list[0]
    left = []
    right = []

    for i in list[1:]:
        if i <= pivot:
            left.append(i)
        else:
            right.append(i)

    return quick_sort(left) + [pivot] + quick_sort(right)


# Merge Sort
def merge_sort(list):
    if len(list) <= 1:
        return list

    mid = len(list) // 2

    left = merge_sort(list[:mid])
    right = merge_sort(list[mid:])

    result = []
    i = 0
    j = 0
```

```
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


# Main Program
list = list(map(int, input("Enter list elements: ").split()))

print("Original List:", list)
print("Quick Sort:", quick_sort(list))
print("Merge Sort:", merge_sort(list))
```

OUTPUT:-

```
y
Enter list elements: 23 12 2 45 67
Original List: [23, 12, 2, 45, 67]
Quick Sort: [2, 12, 23, 45, 67]
Merge Sort: [2, 12, 23, 45, 67]
```

TIME COMPLEXITY

Basis	Quick Sort	Merge Sort
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	$O(n^2)$	O(n log n)

Basis	Quick Sort	Merge Sort
Extra Space	Usually O(log n) for recursion, excluding this implementation's temporary lists	O(n)
Working Method	Selects a pivot and divides elements around it	Divides the list into halves and merges them in sorted order
Stability	Not stable in general	Stable in general

3. Write a program to perform all operations:

A) 1. Stack Using an Array (List)

INPUT:-

```python
stack = []

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter the element: "))
        stack.append(num)
        print("Element pushed")

    elif choice == 2:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Deleted element:", stack.pop())

    elif choice == 3:
        print("Stack:", stack)

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 21
Element pushed

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 23
Element pushed

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Deleted element: 23
```

B)

Queue Using an Array (List)

INPUT:-

```
queue = []

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter the element: "))
        queue.append(num)
        print("Element inserted")

    elif choice == 2:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Deleted element:", queue.pop(0))

    elif choice == 3:
        print("Queue:", queue)

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element: 23
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element: 23
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Deleted element: 23

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue: [23]
```

C) Circular Queue Using an Array

INPUT:-

```python
queue = []
size = 6

front = -1
rear = -1

while True:
    print("1. INSERTION")
    print("2. DELETION")
    print("3.DISPLAY")
    print("4. EXIT")

    num = int(input("Enter your choice: "))

    # insertion
    if num == 1:
        if rear == size - 1:
            print("Queue is Full")
        else:
            value = int(input("Enter element: "))

            if front == -1:
                front = 0

            rear = rear + 1
            queue.append(value)

            print("Element inserted ^^_^^")

    # deletion
    elif num == 2:
```

```
    elif num == 2:
        if front == -1 or front > rear:
            print("Queue is Empty <*_*>")
        else:
            print("Deleted element:", queue[front])
            front = front + 1

    # DISPLAY
    elif num == 3:
        if front == -1 or front > rear:
            print("Queue is Empty <*_*>")
        else:
            print("Queue:", queue[front:rear + 1])

    # EXIT
    elif num == 4:
        break

    else:
        print("Invalid")
```

```python
queue = []
size = 6

front = -1
rear = -1

while True:
    print("1. INSERTION")
    print("2. DELETION")
    print("3.DISPLAY")
    print("4. EXIT")

    num = int(input("Enter your choice: "))

    # insertion
    if num == 1:
        if rear == size - 1:
            print("Queue is Full")
        else:
            value = int(input("Enter element: "))

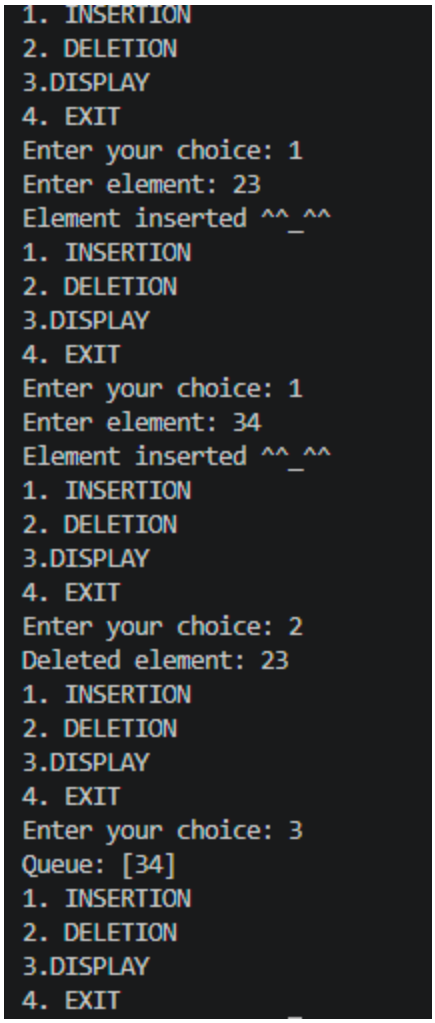
            if front == -1:
                front = 0

            rear = rear + 1
            queue.append(value)

            print("Element inserted ^^_^^")

    # deletion
    elif num == 2:
```

OUTPUT:-



D) Implement Browser History Using Stacks

INPUT:-

```
back_stack = []
forward_stack = []
current_page = "Home"

while True:
    print("\nCurrent Page:", current_page)
    print("1. Visit a new page")
    print("2. Back")
    print("3. Forward")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        page = input("Enter page name: ")
        back_stack.append(current_page)
        current_page = page
        forward_stack = []
        print("Page opened")

    elif choice == 2:
        if len(back_stack) == 0:
            print("No previous page")
        else:
            forward_stack.append(current_page)
            current_page = back_stack.pop()
            print("Moved back")

    elif choice == 3:
        if len(forward_stack) == 0:
            print("No forward page")
```

```
            print("No forward page")
        else:
            back_stack.append(current_page)
            current_page = forward_stack.pop()
            print("Moved forward")

    elif choice == 4:
        print("Browser closed")
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
Current Page: Home
1. Visit a new page
2. Back
3. Forward
4. Exit
Enter your choice: 1
Enter page name: WHATSAPP
Page opened

Current Page: WHATSAPP
1. Visit a new page
2. Back
3. Forward
4. Exit
Enter your choice: 1
Enter page name: GOOGLE
Page opened

Current Page: GOOGLE
1. Visit a new page
2. Back
3. Forward
4. Exit
Enter your choice: 3
No forward page

Current Page: GOOGLE
1. Visit a new page
2. Back
3. Forward
4. Exit
```

4. Write a program to evaluate the Postfix Notation.

INPUT:-

```
stack = []

expression = input("Enter postfix expression (with spaces): ")
tokens = expression.split()

for i in tokens:
    if i.isdigit():
        stack.append(int(i))

    else:
        b = stack.pop()
        a = stack.pop()

        if i == '+':
            stack.append(a + b)
        elif i == '-':
            stack.append(a - b)
        elif i == '*':
            stack.append(a * b)
        elif i == '/':
            stack.append(a / b)

print("Result:", stack.pop())
```

OUTPUT:-

```
Enter postfix expression (with spaces): 5 3 * 4 + 7 /
Result: 2.7142857142857144
```

5. Write a program to convert infix notation into Postfix Notation

INPUT:-

```python
stack = []
postfix = ""

expression = input("Enter infix expression: ")

for i in expression:
    if i.isalnum():
        postfix = postfix + i

    elif i == '(':
        stack.append(i)

    elif i == ')':
        while stack and stack[-1] != '(':
            postfix = postfix + stack.pop()
        stack.pop()

    else:
        while stack and stack[-1] != '(' and (
            (i in '+-' and stack[-1] in '+-*/') or
            (i in '*/' and stack[-1] in '*/')
        ):
            postfix = postfix + stack.pop()

        stack.append(i)

while stack:
    postfix = postfix + stack.pop()

print("Postfix expression:", postfix)
```

OUTPUT:-

```
Enter infix expression: A B + C*
Postfix expression: ABC* +
PS C:\Users\LENOVO\Desktop\python>
```

6. Write a program to perform the operations of the linked list: - Insertion - Deletion – Display

INPUT:-

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("\n1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter the element: "))
        new_node = Node(num)

        if head is None:
            head = new_node
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node

        print("Element inserted")
```

```
elif choice == 2:
    num = int(input("Enter the element to delete: "))

    if head is None:
        print("Linked list is empty")

    elif head.data == num:
        head = head.next
        print("Element deleted")

    else:
        temp = head

        while temp.next is not None and temp.next.data != num:
            temp = temp.next

        if temp.next is None:
            print("Element not found")
        else:
            temp.next = temp.next.next
            print("Element deleted")

elif choice == 3:
    if head is None:
        print("Linked list is empty")
    else:
        temp = head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
            print("No forward page")
        else:
            back_stack.append(current_page)
            current_page = forward_stack.pop()
            print("Moved forward")

    elif choice == 4:
        print("Browser closed")
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the element: 23
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the element: 34
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 2
Enter the element to delete: 23
Element deleted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3
34 -> None
```

7. Write a program to perform the operations of the Circular linked list: - Insertion - Deletion – Display

INPUT:-

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("\n1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter the element: "))
        new_node = Node(num)

        if head is None:
            head = new_node
            new_node.next = head
        else:
            temp = head

            while temp.next != head:
                temp = temp.next

            temp.next = new_node
            new_node.next = head
```

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("\n1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter the element: "))
        new_node = Node(num)

        if head is None:
            head = new_node
            new_node.next = head
        else:
            temp = head

            while temp.next != head:
                temp = temp.next

            temp.next = new_node
            new_node.next = head
```

```
            else:
Click to collapse the range.  next = temp.next.next
                print("Element deleted")

    elif choice == 3:
        if head is None:
            print("Circular linked list is empty")
        else:
            temp = head

            while True:
                print(temp.data, end=" -> ")
                temp = temp.next

                if temp == head:
                    break

            print("(back to head)")

    elif choice == 4:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the element: 23
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the element: 34
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 2
Enter the element to delete: 23
Element deleted
```

8 . Write a program to perform all operations: - For Stack using Linked List. - For Queue using Linked List - For Circular Queue using Linked List

INPUT:-

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


top = None

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter the element: "))
        new_node = Node(num)
        new_node.next = top
        top = new_node
        print("Element pushed")

    elif choice == 2:
        if top is None:
            print("Stack is empty")
        else:
            print("Deleted element:", top.data)
            top = top.next

    elif choice == 3:
```

```
    elif choice == 3:
        if top is None:
            print("Stack is empty")
        else:
            temp = top
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 23
Element pushed

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 34
Element pushed

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
34 -> 23 -> None
```

9.

A) Stack Using Linked List

INPUT:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

top = None

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter element: "))
        new_node = Node(num)
        new_node.next = top
        top = new_node
        print("Element inserted")

    elif choice == 2:
        if top is None:
            print("Stack is empty")
        else:
            print("Deleted element:", top.data)
            top = top.next

    elif choice == 3:
```

```
        temp = top
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 23
Element inserted

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 34
Element inserted

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
34 -> 23 -> None
```

B) Queue Using Linked List

INPUT:-

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

front = None
rear = None

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        num = int(input("Enter element: "))
        new_node = Node(num)

        if front is None:
            front = rear = new_node
        else:
            rear.next = new_node
            rear = new_node

        print("Element inserted")

    elif choice == 2:
        if front is None:
            print("Queue is empty")
```

```
        else:
            print("Deleted element:", front.data)
            front = front.next

            if front is None:
                rear = None

    elif choice == 3:
        temp = front
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:-

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 23
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 45
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Deleted element: 23
```

C) Circular Queue Using Linked List

INPUT:-

```
    if front is None:
        print("Circular queue is empty")
    else:
        print("Deleted element:", front.data)

        if front == rear:
            front = rear = None
        else:
            front = front.next
            rear.next = front

elif choice == 3:
    if front is None:
        print("Circular queue is empty")
    else:
        temp = front

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == front:
                break

        print("(back to front)")

elif choice == 4:
    break

else:
    print("Invalid choice")
```

```
    if front is None:
        print("Circular queue is empty")
    else:
        print("Deleted element:", front.data)

        if front == rear:
            front = rear = None
        else:
            front = front.next
            rear.next = front

elif choice == 3:
    if front is None:
        print("Circular queue is empty")
    else:
        temp = front

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == front:
                break

        print("(back to front)")

elif choice == 4:
    break

else:
    print("Invalid choice")
```

OUTPUT:-

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 12
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 34
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
12 -> 34 -> (back to front)
```

10. Write a program to perform concatenation of two linked lists and Implement Polynomial Representation and Addition using Linked Lists

A) Program to Perform Concatenation of Two Linked List

INPUT:-

```python
class Node:
    def __init__(self, data):
        self.next = None


def display(head):
    temp = head
    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")


# First linked list
head1 = None
tail1 = None

n = int(input("Enter number of elements in first list

for i in range(n):
    data = int(input("Enter element: "))
    new_node = Node(data)

    if head1 is None:
        head1 = tail1 = new_node
    else:
        tail1.next = new_node
        tail1 = new_node


# Second linked list
```

```
84   head2 = None
85   tail2 = None
86
87   n = int(input("Enter number of elements in second list
88
89   for i in range(n):
90       data = int(input("Enter element: "))
91       new_node = Node(data)
92
93       if head2 is None:
94           head2 = tail2 = new_node
95       else:
96           tail2.next = new_node
97           tail2 = new_node
98
99
100  print("First linked list:")
101  display(head1)
102
103  print("Second linked list:")
104  display(head2)
105
106
107  # Concatenation
108  if head1 is None:
109      head1 = head2
110  else:
111      temp = head1
112      while temp.next is not None:
113          temp = temp.next
114      temp.next = head2
```

```
117  print("Concatenated linked list:")
118  display(head1)
```

OUTPUT:-

```
Enter number of elements in first list: 3
Enter element: 12
Enter element: 23
Enter element: 34
Enter number of elements in second list: 2
Enter element: 45
Enter element: 6
First linked list:
12 -> 23 -> 34 -> None
Second linked list:
45 -> 6 -> None
Concatenated linked list:
12 -> 23 -> 34 -> 45 -> 6 -> None
PS C:\Users\LENOVO\Desktop\python>
```

B) Program for Polynomial Representation and Addition Using Linked Lists

INPUT:-

```python
class Node:
    def __init__(self, coefficient, exponent):
        self.coefficient = coefficient
        self.exponent = exponent
        self.next = None


def insert(head, coefficient, exponent):
    new_node = Node(coefficient, exponent)

    if head is None:
        return new_node

    temp = head
    while temp.next is not None:
        temp = temp.next

    temp.next = new_node
    return head


def display(head):
    temp = head

    while temp is not None:
        print(temp.coefficient, "x^", temp.exponent, end="")

        if temp.next is not None:
            print(" + ", end="")

        temp = temp.next
```

```
def display(head):
    print()


def add_polynomials(poly1, poly2):
    result = None

    while poly1 is not None and poly2 is not None:

        if poly1.exponent == poly2.exponent:
            coefficient = poly1.coefficient + poly2.coefficient

            if coefficient != 0:
                result = insert(result, coefficient, poly1.exponent)

            poly1 = poly1.next
            poly2 = poly2.next

        elif poly1.exponent > poly2.exponent:
            result = insert(result, poly1.coefficient, poly1.exponent)
            poly1 = poly1.next

        else:
            result = insert(result, poly2.coefficient, poly2.exponent)
            poly2 = poly2.next

    while poly1 is not None:
        result = insert(result, poly1.coefficient, poly1.exponent)
        poly1 = poly1.next

    while poly2 is not None:
```

```
    def add_polynomials(poly1, poly2):
            result = insert(result, poly2.coefficient, poly2.exponent)
            poly2 = poly2.next

        return result


    # First polynomial
    poly1 = None
    n = int(input("Enter number of terms in first polynomial: "))

    for i in range(n):
        coefficient = int(input("Enter coefficient: "))
        exponent = int(input("Enter exponent: "))
        poly1 = insert(poly1, coefficient, exponent)


    # Second polynomial
    poly2 = None
    n = int(input("Enter number of terms in second polynomial: "))

    for i in range(n):
        coefficient = int(input("Enter coefficient: "))
        exponent = int(input("Enter exponent: "))
        poly2 = insert(poly2, coefficient, exponent)


    print("\nFirst polynomial:")
    display(poly1)

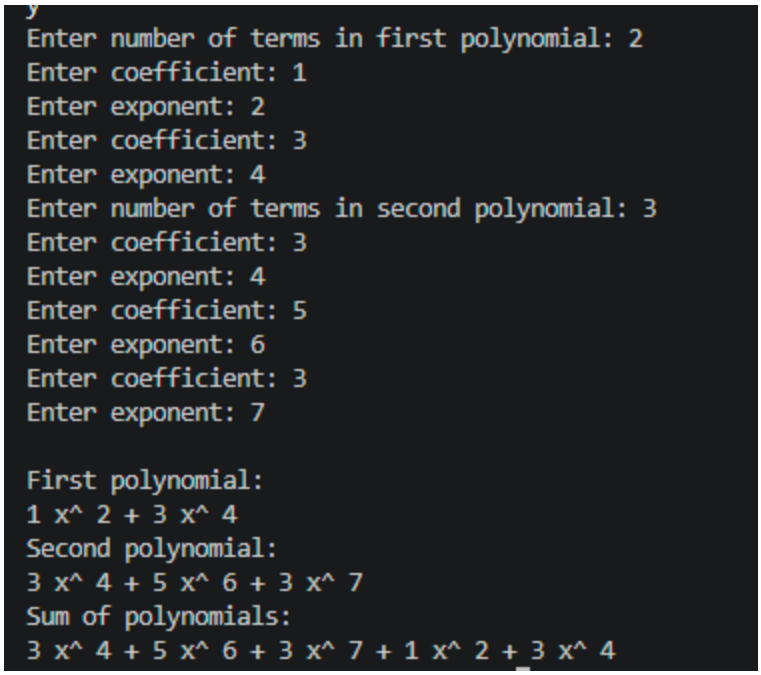
    print("Second polynomial:")
```

```
print("Second polynomial:")
display(poly2)


result = add_polynomials(poly1, poly2)

print("Sum of polynomials:")
display(result)
```

OUTPUT:-


Enter number of terms in first polynomial: 2
Enter coefficient: 1
Enter exponent: 2
Enter coefficient: 3
Enter exponent: 4
Enter number of terms in second polynomial: 3
Enter coefficient: 3
Enter exponent: 4
Enter coefficient: 5
Enter exponent: 6
Enter coefficient: 3
Enter exponent: 7
First polynomial:
1 x^ 2 + 3 x^ 4
Second polynomial:
3 x^ 4 + 5 x^ 6 + 3 x^ 7
Sum of polynomials:
3 x^ 4 + 5 x^ 6 + 3 x^ 7 + 1 x^ 2 + 3 x^ 4

----------END----------